

Microservices

Building Microservice Systems

Sharif University of Technology, Spring 2026

Ali Abrishami

Seyyed Hossein Mirhosseini



How to Model Microservices



Introduction

- The ability to change one microservice in **isolation** from another is vital.
- In essence, microservices are just another form of **modular decomposition**, albeit one that has network-based interaction between the models and all the associated challenges that brings.
- What makes for a good microservice boundary:
 - 1) **Information hiding**
 - 2) **Cohesion**
 - 3) **Coupling**



Information Hiding

- Information hiding describes a desire to hide as many details as possible behind a module.
 1. **Improved development time**: reduce the impact of adding more developers
 2. **Comprehensibility**: module can be looked at in isolation
 3. **Flexibility**: Modules can be changed independently from one another
- The connections between modules are the assumptions which the modules make about each other.



Cohesion

- ***“The code that changes together, stays together.”***
- We’re optimizing our microservice architecture around ease of making changes in **business functionality**.
- So we want to find boundaries within our problem domain that help ensure related behavior is in one place and that communicate with other boundaries as loosely as possible.



Coupling

- When services are loosely coupled, a change to one service should not require a change to another.
- A loosely coupled service **knows as little as it needs** to about the services with which it collaborates.



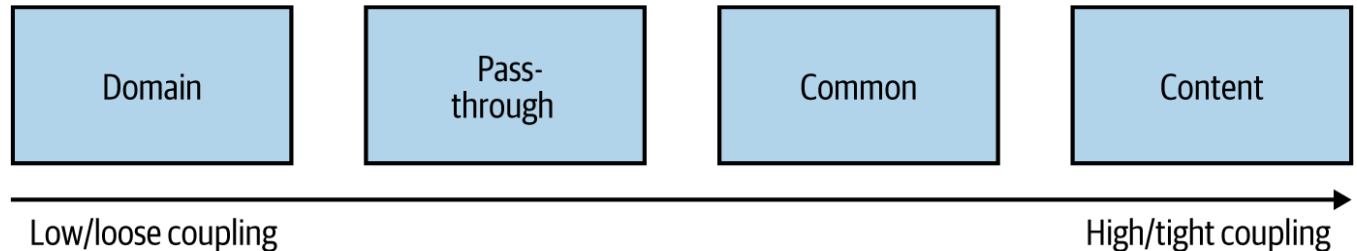
The Interplay of Coupling and Cohesion

- **A structure is stable if cohesion is strong and coupling is low.** (*Larry Constantine*)
- Coupling and cohesion are strongly related and, at some level at least, are arguably the same in that both concepts describe the relationship between things.
- All we can strive to do is to find the **right balance** between these two ideas.



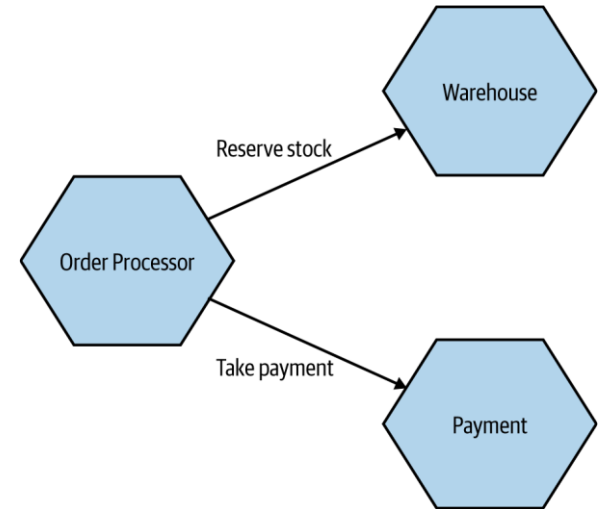
Types of Coupling

- Ultimately, some coupling in our system will be unavoidable.
- What we want to do is **reduce** how much coupling we have.
- Types of coupling:
 1. Domain Coupling
 2. Pass-Through Coupling
 3. Common Coupling
 4. Content Coupling



Domain Coupling

- Domain coupling describes a situation in which one microservice needs to **interact** with another microservice.
- The *Order Processor* is dependent on, and coupled to, the *Warehouse* and *Payment* microservices
- In a microservice architecture, this type of interaction is **largely unavoidable**.



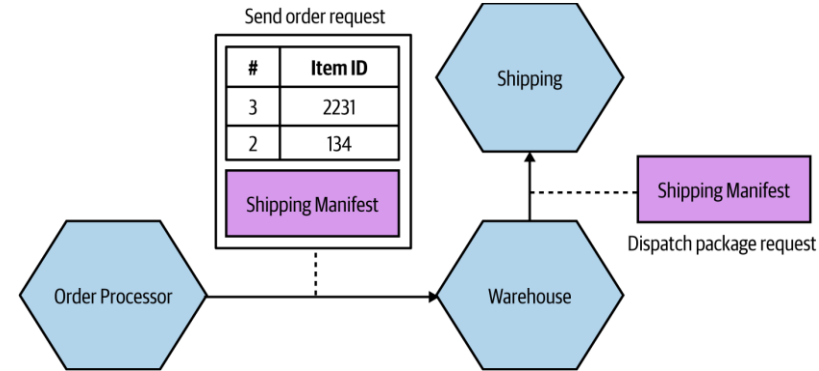
Domain Coupling (cont.)

- Domain coupling is considered to be a **loose form** of coupling.
- A microservice that needs to talk to **lots** of downstream microservices might point to a situation in which too much logic has been **centralized**.
- ***Solution***: Share only what you absolutely have to, and send only the absolute **minimum** amount of data that you need.



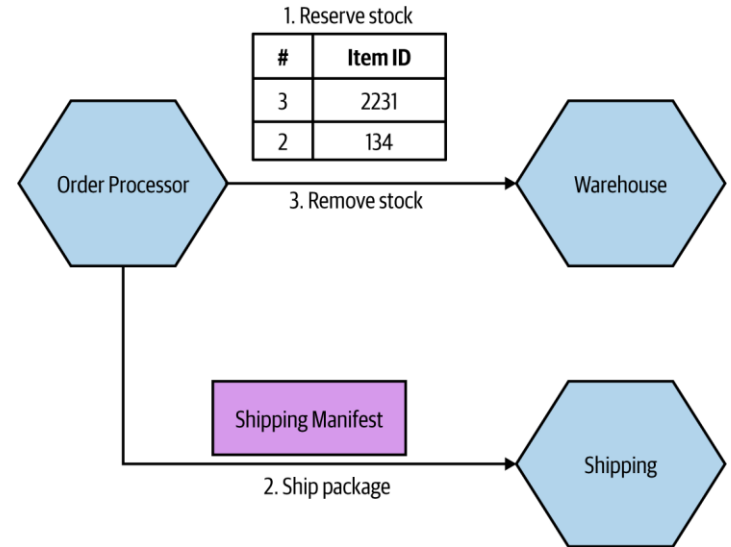
Pass-Through Coupling

- One microservice passes data to another microservice purely because the data is **needed** by some other microservice further downstream.
- It's one of the **most problematic** forms of implementation coupling.
- *data is passed to a microservice purely because another downstream service needs it.*



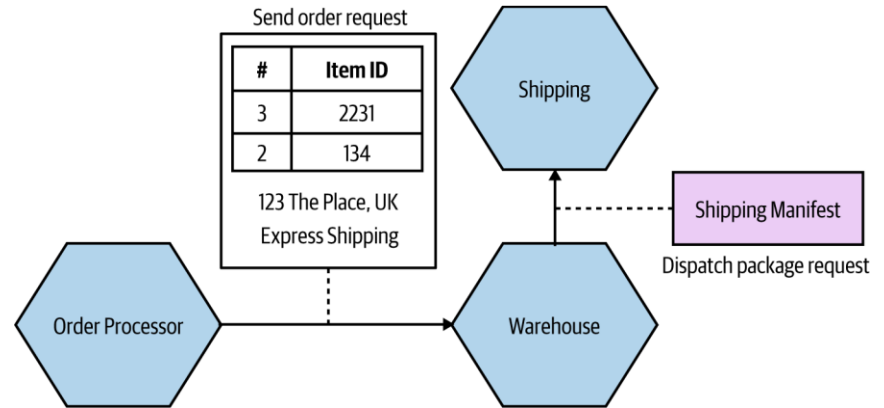
Pass-Through Coupling (cont.)

- **First solution:** The first is to consider whether it makes sense for the calling microservice to just bypass the intermediary.



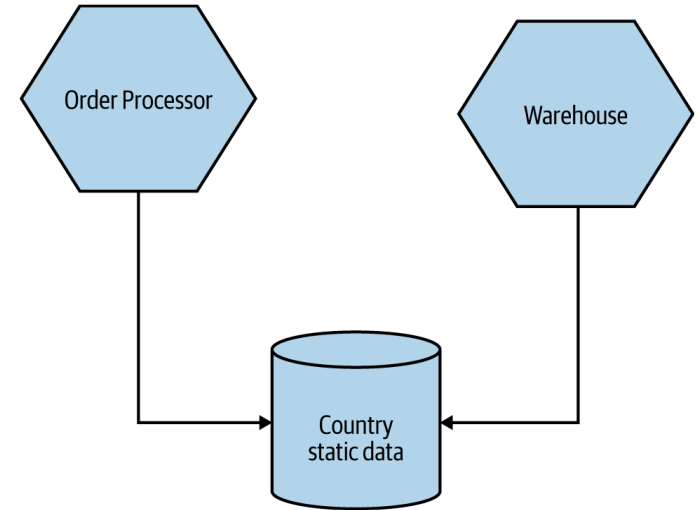
Pass-Through Coupling (cont.)

- **Second solution:** One way to hide this detail would be to have Warehouse take in the required information as part of its contract, and then have it construct the Shipping Manifest locally.



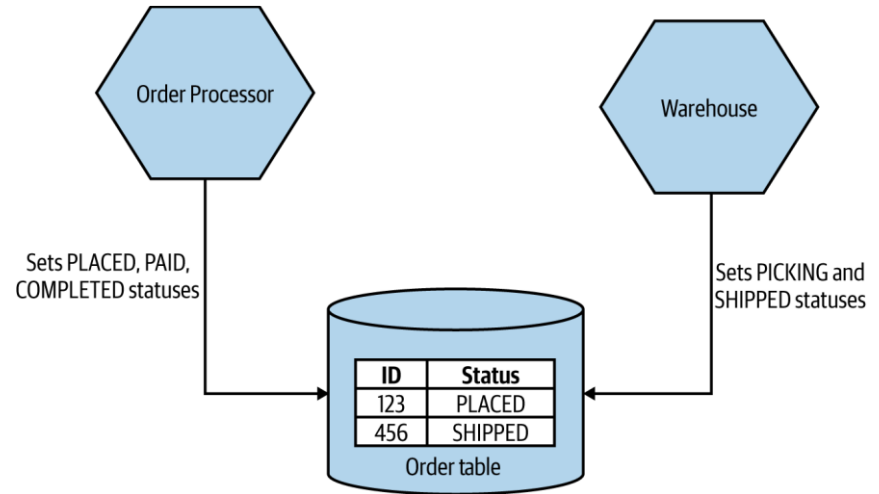
Common Coupling

- Common coupling occurs when two or more microservices make use of a **common set of data**.
- For example, multiple microservices making use of the same **shared database**.
- The main issue with common coupling is that changes to the structure of the data can impact multiple microservices at once.



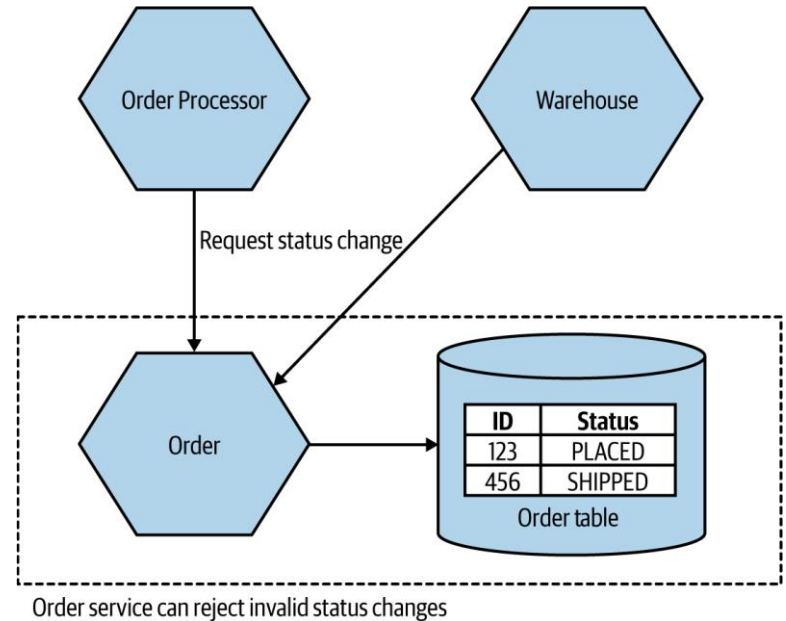
Common Coupling (cont.)

- Common coupling becomes more problematic, though, if the structure of the common data changes more **frequently**.
- *An example of common coupling in which both Order Processor and Warehouse are updating the same order record*



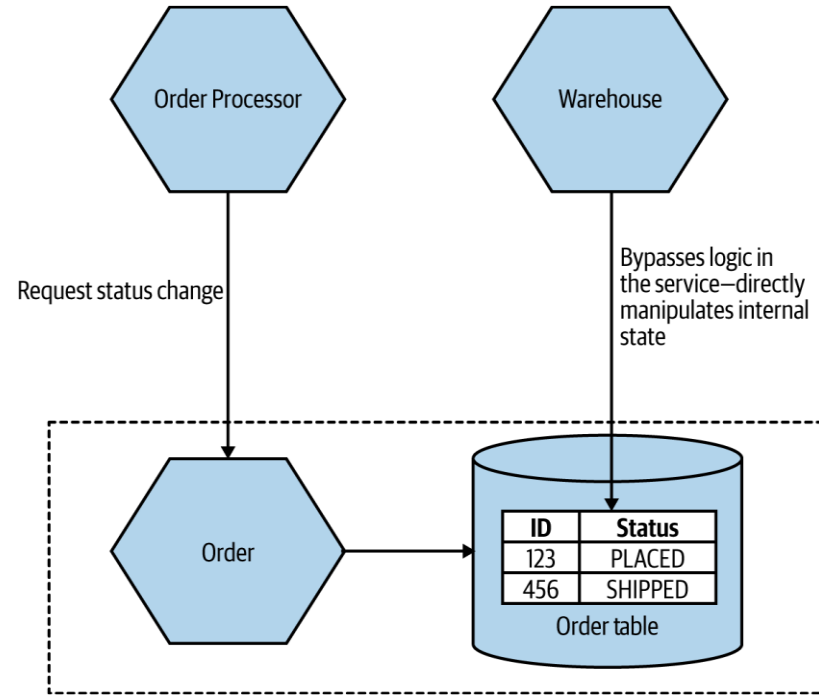
Common Coupling (cont.)

- A potential solution here would be to ensure that a single microservice **manages** the order state.



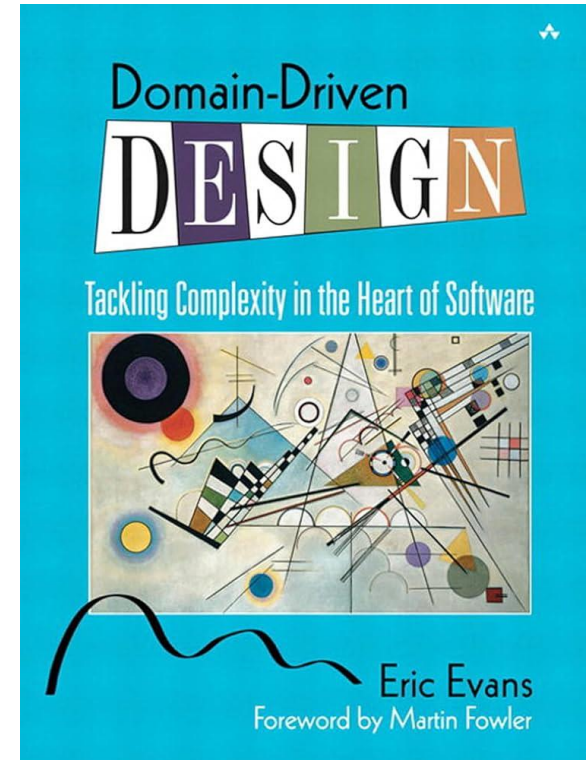
Content Coupling

- Content coupling describes a situation in which an upstream service reaches into the internals of a downstream service and changes its **internal state**.
- **Example:** an external service accessing another microservice's **database** and changing it directly.
- The lines of ownership become less clear, and it becomes more difficult for developers to change a system.



Just Enough Domain-Driven Design

- The primary mechanism we use for finding microservice boundaries is **around the domain** itself, making use of domain-driven design (DDD) to help create a model of our domain.
- Eric Evans's *Domain-Driven Design* presented a series of important ideas that helped us better represent the problem domain in our programs.



Domain-Driven Design

- *Ubiquitous language*
 - Defining and adopting a common language to be used in code and in describing the domain, to aid communication.
- *Aggregate*
 - A collection of objects that are managed as a single entity, typically referring to real-world concepts.
- *Bounded context*
 - An explicit boundary within a business domain that provides functionality to the wider system but that also hides complexity.



Ubiquitous Language

- Ubiquitous language refers to the idea that we should strive to use the same terms in our code as the users use.
- Having a common language between the delivery team and the actual people.
- By working the real-world language into the code, things became much easier.



Aggregate

- Consider an aggregate as a representation of a real domain concept.
 - Something like an Order, an Invoice, a Stock Item, and so on.
- Aggregates typically have a life cycle around them, which opens them up to being implemented as a state machine.
- We want to treat aggregates as self-contained units; we want to ensure that the code that handles the state transitions of an aggregate are grouped together, along with the state itself.
- So one aggregate should be managed by one microservice, although a single microservice might own management of multiple aggregates.



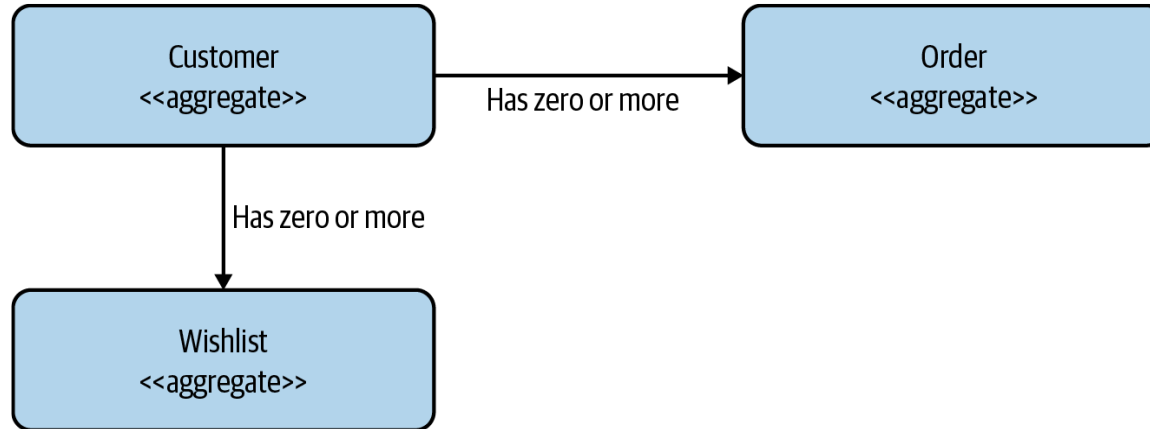
Aggregate (cont.)

- In general, though, you should think of an aggregate as something that has state, identity, a life cycle that will be managed as part of the system.
- Aggregates typically refer to real-world concepts.
- The key thing to understand here is that if an outside party requests a state transition in an aggregate, the aggregate can say no.
- You ideally want to implement your aggregates in such a way that illegal state transitions are impossible.



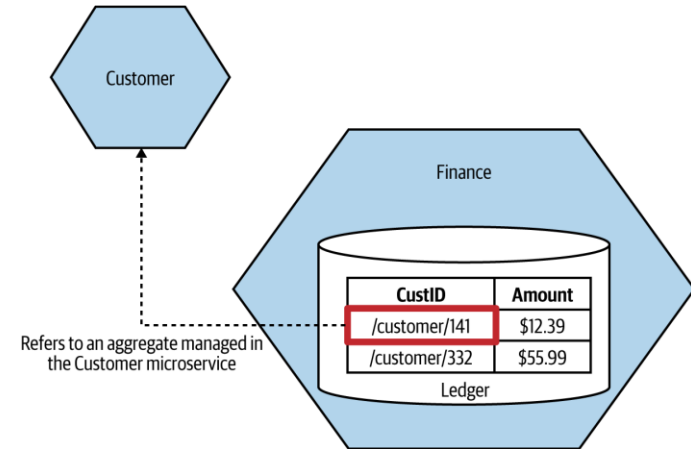
Aggregate (cont.)

- Aggregates can have relationships with other aggregates.



Aggregate (cont.)

- *An example of how a relationship between two aggregates in different microservices can be implemented*
- Rather than a vanilla ID for the customer reference, we instead store a URI, which we might use if building a REST-based system.



Bounded Context

- A *bounded context* typically represents a larger organizational boundary.
- Bounded contexts hide implementation detail.
- From an implementation point of view, bounded contexts contain one or more aggregates. Some aggregates may be exposed outside the bounded context; others may be hidden internally.
- For MusicCorp, we can consider the **finance department** and the **warehouse** to be two separate bounded contexts.
- Hidden models
- Shared models

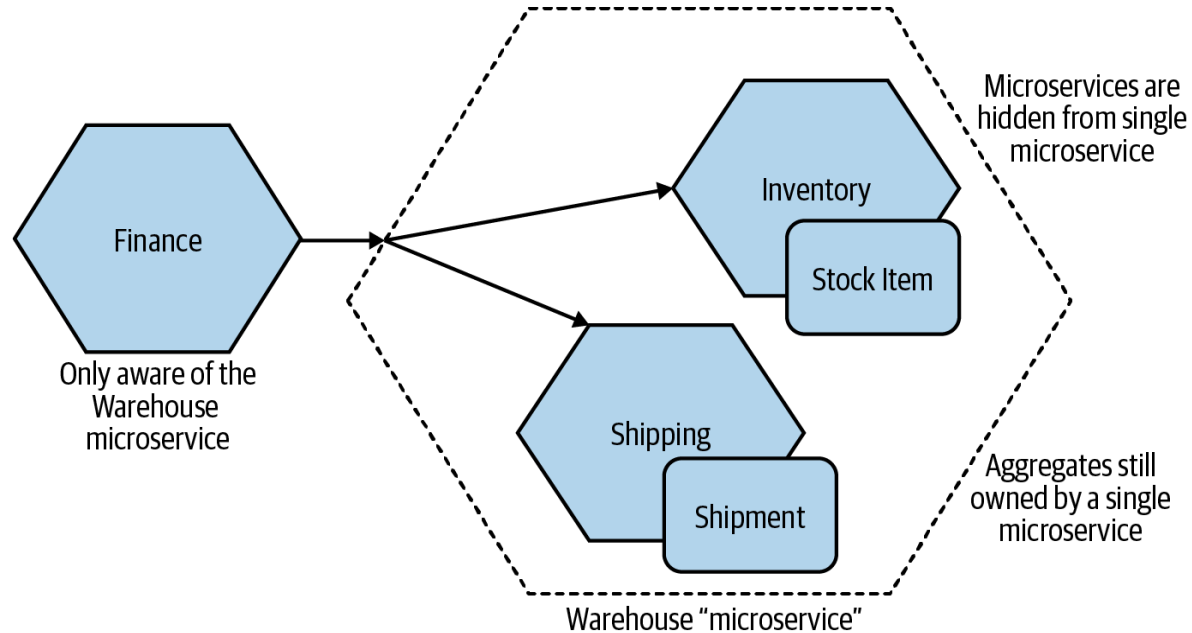


Mapping Aggregates and Bounded Contexts to Microservices

- The aggregate is a self-contained state machine that focuses on a single domain concept in our system, with the bounded context representing a **collection of associated aggregates**, again with an explicit interface to the wider world.
- One microservice can manage one or more aggregates, but we **don't want** one aggregate to be managed by more than one microservice.
- When considering the boundaries of your microservices, first think in terms of the larger, **coarser-grained contexts**, and then subdivide along these nested contexts when you're looking for the benefits of splitting out these seams.



Mapping Aggregates and Bounded Contexts to Microservices



Event Storming

- Event storming is a collaborative brainstorming exercise designed to help surface a domain model.
- Event storming brings together technical and nontechnical stakeholders in a joint exercise.
- The idea is that, by making the development of the domain model a joint activity, you end up with a shared, joined-up view of the world.



Event Storming (cont.)

- Firstly, get everyone in a room together.
- You want representatives of all parts of the domain that you plan to model: users, subject matter experts, product owners—whoever is best placed to help represent each part of the domain.
- The main modeling tool is sticky notes to capture the various concepts, with differently colored notes representing different concepts.



Event Storming (cont.)

- The exercise starts with the participants identifying the *domain events*. they are the facts that you care about. These are captured on orange sticky notes.
 - “Order Placed”
 - “Payment Received.”
- Next, participants identify the commands that cause these events to happen.
- A command is a decision made by a human (a user of the software) to do something.
- Here you are trying to understand the system’s boundary, and identify the key human actors in the system.
- Commands are captured on blue sticky notes.



Event Storming (cont.)

- With events and commands captured, aggregates come next.
- Aggregates are represented by yellow sticky notes, and the commands and events associated with that aggregate are moved and clustered around the aggregate.
- With the aggregates identified, they are grouped into bounded contexts. Bounded contexts most commonly follow a company's organizational structure.



The Case for Domain-Driven Design for Microservices

- Firstly, a big part of what makes DDD so powerful is that **bounded contexts**.
- Secondly, the focus on defining a common, **ubiquitous language** helps greatly when it comes to defining microservice endpoints.
- If our systems are **decomposed** along the bounded contexts that represent our domain, any changes we want to make are more likely to be isolated to a single microservice boundary.
- Fundamentally, DDD puts the **business domain** at the **heart of the software** we are building.



Alternatives to Business Domain Boundaries

- DDD can be incredibly useful when building microservice architectures.
- But it would be a mistake to think that this is the **only technique** you should consider when finding microservice boundaries.
- In fact, I often use multiple methods in conjunction with DDD to help identify how (and if) a system should be split:
 - Volatility
 - Data
 - Technology
 - Organization



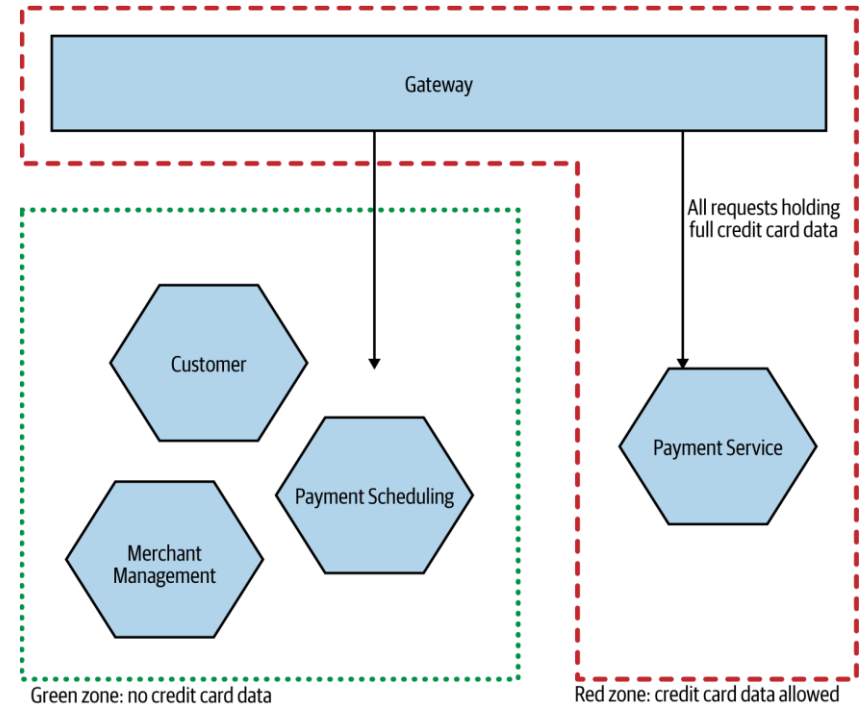
Volatility

- Volatility-based decomposition has you identify the parts of your system going through **more frequent change** and then extract that functionality into their own services.
- Mode 1 systems, we are told, don't change much and don't need much business involvement.
- Mode 2 is where the action is, with systems that need to change quickly and that require close involvement from the business.
- This technique is highly useful in helping to determine boundaries if the main driver is about **fast time to market**—extracting functionality that is changing or needs to **change frequently** makes perfect sense in such a situation.



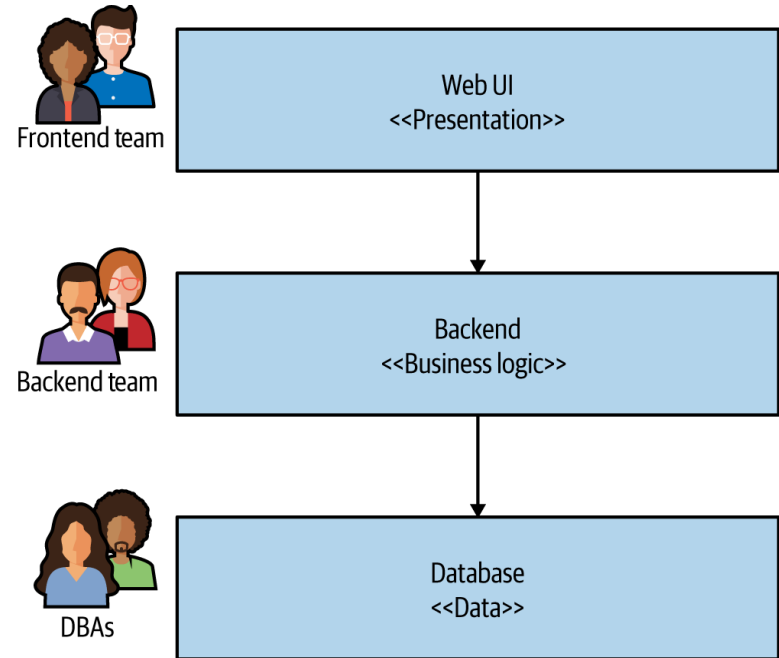
Data

- The nature of the data you hold and manage can drive you toward different forms of decomposition.
- **Segregation of data** is often driven by a variety of privacy and security concerns.



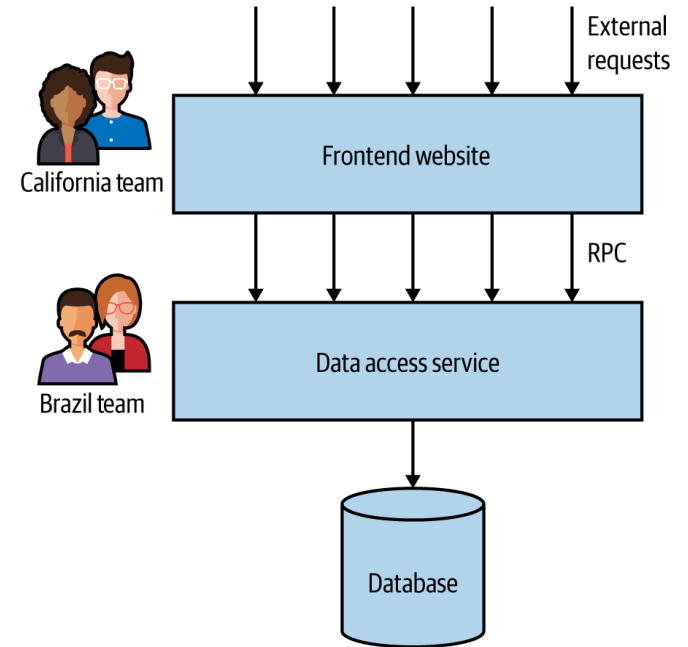
Technology

- The need to make use of different **technology** can also be a factor in terms of finding a boundary.



Organizational

- Shared ownership of microservices is a fraught affair.
- **Example:** A service boundary split across technical seams.



Comparison

- Volatility-based decomposition can make a lot of sense if your focus is on improving the **speed of delivery**, but if this causes you to extract a service that crosses organizational boundaries, then expect your pace of change to suffer due to delivery contention.
- Organizational and domain-driven service boundaries are a good **starting point**.
- The ideas presented in Eric Evans's *Domain-Driven Design* are very useful to us in finding sensible boundaries for our services.



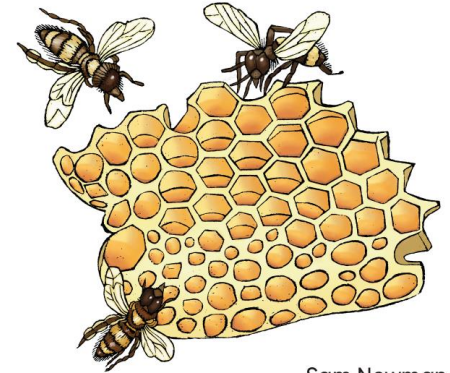
References

- Building Microservices: Designing Fine-Grained Systems,
Sam Newman
 - Chapter 2

O'REILLY®

Building Microservices

Designing Fine-Grained Systems



Sam Newman

Second
Edition

